

Programmierbarer EPROM-Simulator

'Weiche' Firmware

Hans Lotter

Beim Erstellen von EPROM-Software, die zum Beispiel einem Steuerrechner 'sagen' soll, was er zu tun hat, ergibt sich fast immer ein Problem: Nachdem man das EPROM 'gebrannt' hat, läuft das Programm nicht so, wie es soll. Die Konsequenz daraus ist, daß man die Software am Entwicklungsrechner ändert, ein neues EPROM programmiert und wieder testet. Mit dem EPROM-Simulator PEPS kann man diese Arbeiten erheblich vereinfachen.

PEPS ist ein Instrument zum Entwickeln von Software, die in einem EPROM abgespeichert werden soll. Dabei übernimmt PEPS (Programmierbarer EPROM-Simulator) die Funktion des EPROMs und erlaubt so das schnelle Austesten und Ändern der Software.

Während der Software-Entwicklungsphase ist PEPS ständig mit dem Entwicklungs- und dem Testsystem verbunden. Das Programm wird vom Entwicklungssystem seriell in das interne RAM von PEPS eingeschrieben. Dazu ist lediglich ein Port mit vier freien Bits am System nötig. Nach der Datenübertragung entspricht PEPS einem normalen programmierten EPROM.

PEPS kann vier verschiedene EPROM-Typen simulieren: 2716, 2732, 2764 und 27128. Entsprechend dem zu simulierenden EPROM-Typ ist die PEPS-Platine mit RAM-Bausteinen (6116 oder 6264) zu bestücken. Die Stromversorgung des Simulators kann wahlweise aus dem Test- oder dem Entwicklungssystem erfolgen. Die Stromaufnahme von PEPS ist nur etwa halb so groß wie die

des simulierten EPROMs. Für spezielle Anwendungen kann man PEPS mit einem Akku oder einer Batterie puffern, wodurch man die einmal eingeschriebenen Daten über einen langen Zeitraum im Simulator halten kann.

Angeschlossen

PEPS wird über ein 24- oder 28poliges Flachbandkabel mit dem EPROM-Sockel verbunden. Man kann aber auch einen DIL-Stecker unter die Fassung N1 löten und die PEPS-Platine direkt in die EPROM-Fassung stecken.

Die Verbindung zum Entwicklungssystem geschieht über den zehnpoligen Stecker N2. Es genügt, die vier Steuerleitungen Clock, M0, M1 und Data sowie eine Masseleitung anzuschließen. Die 5-V-Versorgungsspannung ist nur dann erforderlich, wenn das Entwicklungssystem die Stromversorgung übernehmen soll. Die Steuerleitungen kann man an vier beliebigen Ausgangsleitungen eines freien Parallel-Ports (z. B. Z80-PIO) oder einer Centronics-Schnittstelle anschließen. Dabei muß

allerdings gewährleistet sein, daß die Daten zwischen den einzelnen Strobe-Impulsen stabil bleiben, was normalerweise auch der Fall ist.

11 ICs

Die zwei RAM-Bausteine Z8 und Z9 bilden den Kern der Schaltung (Bild 1). Diese ICs werden über die vier umschaltbaren Multiplexer IC2 bis IC5 entweder vom EPROM-Sockel (N1) oder von den beiden Zahlern IC1 und IC4 adressiert. Das CMOS-Schieberegister IC11 nimmt die seriellen Daten vom Entwicklungssystem auf und dient gleichzeitig als Zwischenspeicher beim späteren Einschreiben der Daten in das RAM. Das Umschalten zwischen den verschiedenen Be-

triebsarten von PEPS geschieht über den Dekoder IC10. Der Tristate-Treiber IC7 trennt den internen Datenbus von den Datenleitungen des EPROM-Sockels.

Wenn man es wünscht, kann man PEPS mit einem Akku (Batterie) puffern. In diesem Fall bleiben die Daten im RAM auch nach dem Abschalten der Versorgungsspannung erhalten. Zum Schutz des RAMs bei Spannungsabfall dienen die Transistoren T1 bis T3. Bei der Pufferung sollte man in jedem Fall RAM-Bausteine mit niedriger Stand-by-Stromaufnahme wählen (z. B. 6116LP-3).

Modifiziert

Für normale Anwendungen, also ohne Akku-/Batteriepufferung, braucht man einige Bauteile nicht zu bestücken. Es genügt, neben der Bestückung der ICs die Bauteile C1, C2, C4 bis C6, R12 sowie die Leuchtdiode einzusetzen. Die Diode D2, der Widerstand R8 sowie die Kollektor/Emitter-Strecke von T1 bis T3 sind durch Drahtbrücken zu ersetzen. Der Kondensator C3, die übrigen Widerstände und die Diode D1 entfallen.

Mit den Jumpern J1 bis J3 kann man PEPS für verschiedene EPROM-Typen 'einstellen'. Je nach Speicherkapazität des zu simulierenden EPROMs muß man als Speicher RAMs des Typs 6116 (2 KByte) oder 6264 (8 KByte) verwenden. Welches IC für das jeweilige EPROM einzusetzen ist, zeigt Tabelle 1. Verwendet man die 6116-RAMs, müssen die ICs so eingesetzt werden, daß die Pins 1, 2, 27 und 28 der Fassung frei bleiben. Der Tabelle 1 kann man ebenfalls entnehmen, wie die Jumper J1 bis J3 zu setzen sind, wobei man beachten muß, daß J1 durch eine Verbindung auf der Platine geschlossen ist. Die übrigen Jumper J4 bis J6 sind für spätere Erweiterungen vorgesehen und dürfen nicht gebrückt werden.

EPROM-Typ	2716 (2K)	2732 (4K)	2764 (8K)	27128 (16K)
Jumper J1	X	X	5V	5V
Jumper J2	5V	5V	offen	offen
Jumper J3	A, D	A, C	B, D	B, D
Socket Z8	6116	6116	6264	6264
Socket Z9	leer	6116	leer	6264

Tabelle 1. Die verschiedenen Versionen des Simulators. In der Tabelle bedeutet 'X', daß der Jumper keine Funktion hat. '5V' zeigt an, daß die 5-V-Versorgungsspannung von PEPS bei geschlossenem Jumper aus dem Testgerät erfolgt.

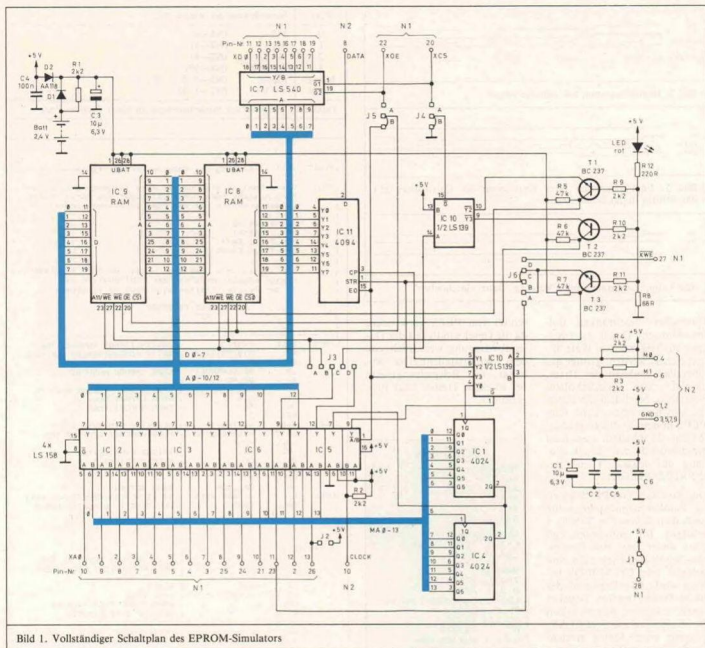


Bild 1. Vollständiger Schaltplan des EPROM-Simulators

Gesteuert

PEPS kennt vier verschiedene Betriebsarten (Tabelle 2), die man durch M0 und M1 einstellt. Die 'Clock'-Leitung (aktiv high) löst dann die durch M0 und M1 vorgewählte Funktion aus. Über die serielle 'Data'-Leitung wird im Modus 1 das vom Entwicklungssystem gesendete Daten-Byte eingelesen.

Um ein Daten-Byte an eine beliebige Adresse im PEPS-RAM zu schreiben, sind drei Schritte erforderlich.

Adresse einstellen (Bild 2): Im Modus 3 setzt ein Low-Pegel am Clock-Eingang den inter-

nen Adreßzähler auf Null. Im Modus 0 wird dann die eingestellte Adresse bei jeder positiven Clock-Flanke um den Wert Eins erhöht. Diese einmal eingestellte Adresse bleibt auch beim Modus-Wechsel erhalten.

Daten-Byte einschieben (Bild 3): Für diese Funktion muß der Modus 1 gewählt werden. Anschließend wird das Byte seriell

über die 'Data'-Leitung übertragen. Bei jeder negativen Clock-Flanke wird der jeweilige Pegel am Data-Eingang als Bit übernommen, wobei das Bit 0 immer zuletzt gesendet werden muß. Ein einmal eingeschobenes Byte bleibt auch beim Modus-Wechsel erhalten. Bild 3 zeigt das Impulsdiagramm beim Einschieben des Bytes 35h (00110101b).

Daten einschreiben (Bild 4): Für diese Funktion ist der Modus 2 einzustellen. Das im Modus 1 eingeschobene Byte wird nun mit einem positiven Impuls am Clock-Eingang an die im Modus 0 eingestellte Adresse geschrieben.

Abschließend schaltet man PEPS in den Mode 3, wodurch das EPROM simuliert wird.

Zur Beschleunigung der Übertragung lassen sich einige Schritte abkürzen. Da normalerweise die Daten blockweise übertragen werden, kann man das Einstellen der Adresse auf einfaches Hochzählen der Adresse nach jedem gesendeten

M1	M0	Modus
0	0	Adresse hochzählen (Modus 0)
0	1	Daten einschieben (Modus 1)
1	0	Daten einschreiben (Modus 2)
1	1	EPROM simulieren (Modus 3)

Tabelle 2. Die vier Betriebsmodi von PEPS

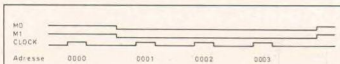


Bild 2. Impulsdiagramm bei 'Adresse setzen'

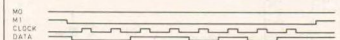


Bild 3. Impulsdiagramm beim Einschreiben des Daten-Bytes 35h (1010101b) in das Schieberegister

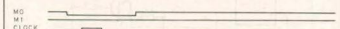


Bild 4. Impulsdiagramm beim Vorgang 'Daten einschreiben'

Daten-Byte beschränken. Sollen identische Daten, zum Beispiel ein Feld mit dem Wert '0', gespeichert werden, genügt das einmalige Einlesen des Bytes, gefolgt von wiederholtem Schreiben. Nachdem die Daten im Speicher stehen, kann man PEPS in den 'Simulier-Modus' (Modus 3) schalten. Legt man anschließend den Clock-Eingang auf logisch 1, ist das 'EPROM' aktiviert.

Der Anschluß von PEPS an eine Parallel-Schnittstelle sollte nach dem Schema in Tabelle 3 erfolgen. Im einfachsten Fall kann dieser Port eine Centronics-Schnittstelle sein, die zum Beispiel vom CP/M-BIOS bedient wird. Allerdings muß der BIOS-Treiber immer 'Drucker bereit' erkennen, was oft schon durch eine Leitung vom BUSY-Eingang nach Masse erreicht werden kann. Wer in seinem System einen freien Port hat, sollte diesen auch für den An-

schluß von PEPS verwenden, da die Geschwindigkeit der Datenübertragung wesentlich von dem Schnittstellentreiber abhängt. Ein Beispielprogramm für so einen Treiber zeigt Programm 1. □

Stückliste

Widerstände

R1,2,3,4 2k2
R5,6,7 47k
R8 68R
R12 220R

Kondensatoren

C1,3 10µF/6,3V Tantal
C2,4,5,6 100nF

Halbleiter

T1,2,3 BC237
D1,2 AA118
LED Leuchtdiode 3 mm, rot
IC1,4 CD4024
IC2,3,5,6 74LS158
IC7 74LS540
IC8,9 6116 oder 6264, je nach EPROM
IC10 74LS139
IC11 CD4094

Port	Steuerleitung an Stecker N2
0	M0 (N2-4)
1	M1 (N2-6)
2	DATA (N2-8)
3	CLOCK (N2-10)
Masse +5V	Masse (N2-3, 5, 7, 9) (N2-1, 2)

Tabelle 3. Anschluß der Steuerleitungen an einen Rechner-Port

```

;
;PINIT Initialisierung des Ausgabe-Ports, falls notwendig.
;POUT Diese Routine gibt das im Akku stehende Byte an den
;      PEPS Ausgabe Port. Die Leitungen des Ports messen
;      folgendermaßen mit dem PEPS verbunden werden:
;
;      Bit 0 mit M0
;      Bit 1 mit M1
;      Bit 2 mit Data
;      Bit 3 mit Clock
;
;      Die Zeit, die benötigt wird, um PEPS zu laden,
;      hängt von der POUT-Routine ab.
;      In den meisten Fällen wird POUT so aussehen:
;
;      OUT PEPSPORT
;      RET

```

```

;
;LPEPS: CALL PINIT ; INITIALISIERE AUSGABE PORT
;        LD A,00001011B ; SETZE ADRESSZÄHLER AUF NULL
;        CALL POUT ;
;        LD HL,(LAENGE) ; LADE LAENGE NACH DE
;        EX DE,HL ;
;        LD HL,(START) ; LADE START-ADRESSE NACH HL
;        LD A,D ; WENN LAENGE NULL,
;        OR E ; DANN SOFORT ENDE
;        Z ;
;        LOOP: WRITEBYTE ; SCHREIBE EIN BYTE EIN
;        LD A,00001000B ; PEPS ADRESSE EINS WEITER
;        CALL POUT ;
;        INC DE ; ZEIGER EINS WEITER
;        DEC HL ; ENDE ERREICHT ?
;        LD A,D ;
;        OR E ;
;        JP NZ,LOOP ; DANN RETURN
;        LD A,00001011B ; SETZE ADRESSZÄHLER AUF NULL
;        CALL POUT ; UND GEHE IN SIMULIER-MODUS
;        RET ;
;
;
;WRITEBYTE: ;
;           ; LADE (HL) IN PEPS. HL UND
;           ; DE WERDEN NICHT VERÄNDERT
;
;           LD B,B ; 8 BIT
;           LD A,(HL) ; LADE BYTE
;           CPL ; KOMPLEMENT FUER 74LS540
;           RLCA ; SCHIEBE BYTE RICHTIG
;           LD C,A ; RETTE IN C
;
;           LD A,C ;
;           LD RLCA ;
;           LD A,D ;
;           AND 00001000B ;
;           OR 00001001B ;
;           CALL POUT ;
;           AND 00001111B ;
;           CALL POUT ;
;           DEC B ;
;           JP NZ,WRITE ;
;           LD A,00001010B ;
;           CALL POUT ;
;           XOR A ;
;           CALL POUT ;
;           RET ;
;
;
;PORT INITIALISIEREN
;KEINE INITIALISIERUNG
;
;PORT AUSGABE
;OUT (PEPSPORT),A
;RET

```

Programm 1. Beispiel für einen Schnittstellentreiber in Z80-Mnemonics. Das Programm geht davon aus, daß die Länge des zu übertragenden Speicherbereichs in einer Speicherzelle 'LAENGE' steht und die Startadresse des Bereichs in 'START'.

